

Natural Language	Skill Invoked
------------------	---------------

Confluence Guide: Installation & Use Cases

Table of Contents

- 1. What is BMAD?
- 2. Core Concepts
- 3. Installation Guide
- 4. Project Structure After Installation
- 5. Modules Overview
- 6. AI Agents (Personas)
- 7. Skills Catalog
- 8. Full Development Workflow
- 9. Use Cases & Examples
- 10. Configuration Reference
- 11. IDE Integration – GitHub Copilot
- 12. Folder Conventions & Outputs
- 13. FAQ & Troubleshooting

1. What is BMAD?

BMAD (Build with AI-Driven Agile Development) is an open-source meta-framework that supercharges AI coding assistants (GitHub Copilot, Claude, Codex, etc.) with a full software delivery methodology — from idea to deployed code.

Instead of using an AI assistant only for code auto-completion, BMAD gives the assistant **structured roles, skills, and a repeatable workflow** so it can act as:

- A **Business Analyst** gathering requirements
- A **Product Manager** creating PRDs
- An **Architect** designing technical solutions
- A **Developer** writing and reviewing code
- A **QA Engineer** generating tests
- A **Scrum Master** managing sprints

2. Core Concepts

Concept	Description
Skill	A named, reusable capability loaded into the AI at runtime (e.g., bmad-create-prd). Each skill has a SKILL.md file with detailed instructions.

Agent	A persona with a specific role, personality, and set of skills (e.g., John = Product Manager, Amelia =
	ons, patterns, and
Artifacts	Documents produced during a workflow: Product Briefs, PRDs, Architecture docs, Epics/Stories, Sprint plans, and code.

3. Installation Guide

Prerequisites

Requirement	Details
Node.js	v18 or later (for the BMAD CLI installer)
CLI	GitHub Copilot CLI, Claude Code, OpenAI Codex
Git	Repository initialized
OpenAI - ChatGPT Enterprise	License Required

Step-by-Step Installation

```
# Command to install codex CLI
npm install -g @openai/codex

# Command to install claude code
npm install -g @anthropic-ai/claude-code

# Command to install github copilot CLI
npm install -g @github/copilot

# Run from the root of your project repository
npx bmad-method@latest install
```

The interactive wizard will ask:

- 1. **IDE selection** – choose github-copilot, claude-code, or codex
- 2. **Modules** – select core (always included) and bmm (full agile workflow)
- 3. **Project name** – e.g., hw-backend-service
- 4. **User name** – your display name
- 5. **User skill level** – beginner, intermediate, or expert

What the installer does

- 1. Creates the `_bmad/` directory with all skill definitions and agent configurations.
- 2. Creates the `_bmad/_config/manifest.yaml` tracking installed modules and IDEs.
- 3. Generates IDE-specific configuration files (e.g., `.github/copilot-instructions.md` for GitHub Copilot).
- 4. Creates the `_bmad-output/` directory for all generated artifacts.
- 5. Registers skills in `.github/skills/` so GitHub Copilot can discover and invoke them.

4. Project Structure After Installation

your-project/

├── github-copilot.yaml	← IDE-specific settings
├── core/	← Core utility skills
├── bmm/	← BMad Method module (full workflow)
│ ├── 1-analysis/	← Research & analysis skills
│ ├── 2-plan-workflows/	← Planning & PRD skills
│ ├── 3-solutioning/	← Architecture & epics skills
│ └── 4-implementation/	← Dev, QA, sprint skills
├── _bmad-output/	← All AI-generated artifacts (commit this)
│ ├── project-context.md	← LLM-optimized codebase rules
│ ├── planning-artifacts/	← PRDs, architecture docs, epics/stories
│ └── implementation-artifacts/	← Sprint plans, story files, test suites
└── .github/	
└── skills/	← GitHub Copilot skill entry points

5. Modules Overview

core module – Utility Skills

General-purpose skills that work independently of any project or workflow phase.

Skill	Trigger Phrase	Purpose
bmad-help	bmad help	Recommends next skill or answers BMAD questions
bmad-init	(automatic)	Loads configuration variables for other skills
bmad-brainstorming	help me brainstorm	Facilitated ideation sessions
bmad-distillator	distill documents	Compresses large docs into LLM-optimized summaries
bmad-advanced-elicitation	(after any output)	Deepens AI critique using Socratic/pre-mortem methods
bmad-editorial-review-prose	review for prose	Copy-edits any document for clarity
bmad-editorial-review-structure	editorial review of structure	Restructures documents for better organization
bmad-review-adversarial-general	critical review	Cynical adversarial review of any content
bmad-review-edge-case-hunter	(after PRDs/code/specs)	Finds unhandled edge cases and boundary conditions

Individual reflection	Individual reflection	Group discussion

bmm module – Full Agile Workflow (BMad Method)

Structured skills organized across a 4-phase delivery lifecycle.

Phase	Phase Name	Purpose
1	Analysis	Research, domain understanding, product brief creation
2	Plan & Workflows	PRD creation, UX design, requirement validation
3	Solutioning	Architecture, epics & stories, readiness checks
4	Implementation	Sprint planning, development, code review, QA, retrospectives

6. AI Agents (Personas)

BMAD agents are named personas with specialized expertise. You can invoke them by name in GitHub Copilot chat.

BMM Module Agents

Icon	Name	Alias	Role	How to Invoke
	Mary	Analyst	Strategic Business Analyst	talk to Mary or business analyst
	John	PM	Product Manager	talk to John or product manager
	Sally	UX	UX Designer	talk to Sally or UX designer
	Winston	Architect	System Architect	talk to Winston or architect
	Amelia	Dev	Senior Developer	talk to Amelia or developer agent
	Quinn	QA	QA Engineer	talk to Quinn or QA engineer
	Bob	SM	Scrum Master	talk to Bob or scrum master
	Barry	Solo Dev	Quick Flow Developer	talk to Barry or quick flow solo dev
	Paige	Tech Writer	Technical Writer	talk to Paige or tech writer

Agent Capabilities

Mary – Business Analyst

- Market research, competitive analysis, requirements elicitation

- Translates vague ideas into actionable specifications
- Uses Porter's Five Forces, SWOT, Root Cause Analysis

Follows code to be done framework

Garry – UX Designer

- User research, interaction design, UI patterns
- Creates wireframe descriptions and UX specifications
- Balances user empathy with functional edge cases

Winston – System Architect

- Distributed systems, cloud infrastructure, API design
- Technology selection with trade-off analysis
- Produces architecture decision records (ADRs)

Amelia – Developer

- Test-driven story execution
- Strict adherence to coding standards and story acceptance criteria
- Every task covered by unit tests before marking complete

Quinn – QA Engineer

- API and E2E test generation for existing features
- Practical coverage-first approach
- Tests expected to pass on first run

Bob – Scrum Master

- Sprint planning and story preparation
- Agile ceremonies and backlog management
- Zero tolerance for ambiguity in acceptance criteria

Barry – Quick Flow Solo Dev

- Combines spec creation + implementation in one pass
- Minimum ceremony, maximum output
- Best for greenfield features or solo development

Paige – Technical Writer

- Documentation creation with Mermaid diagrams
 - Validates documents against standards
 - Explains complex concepts with analogies and visuals
-

7. Skills Catalog

Phase 1 – Analysis Skills

			enerate useful
bmad-generate-project-context	GPC	Scan codebase and generate LLM-optimized project-context.md	
bmad-product-brief	CB	Guided discovery to define and refine a product idea	
bmad-domain-research	DR	Industry domain deep dive and terminology mapping	
bmad-market-research	MR	Market analysis, competitive landscape, customer needs	
bmad-technical-research	TR	Technical feasibility, architecture options, trade-offs	
bmad-brainstorming	BP	Facilitated creative brainstorming using multiple techniques	

Phase 2 – Planning Skills

Skill	Menu Code	Description
bmad-create-prd	CP	Expert-facilitated Product Requirements Document creation
bmad-validate-prd	VP	Validate PRD against completeness and quality standards
bmad-edit-prd	EP	Revise an existing PRD based on feedback or new requirements
bmad-create-ux-design	CU	Create UX specifications and interaction design guidelines

Phase 3 – Solutioning Skills

Skill	Menu Code	Description
bmad-create-architecture	CA	Document technical decisions and system design
bmad-create-epics-and-stories	CE	Break requirements into epics and user stories
bmad-check-implementation-readiness	IR	Validate PRD, UX, Architecture, and Epics are aligned and complete
bmad-generate-project-context	GPC	Create/update the AI-rules context file from codebase scan

Phase 4 – Implementation Skills

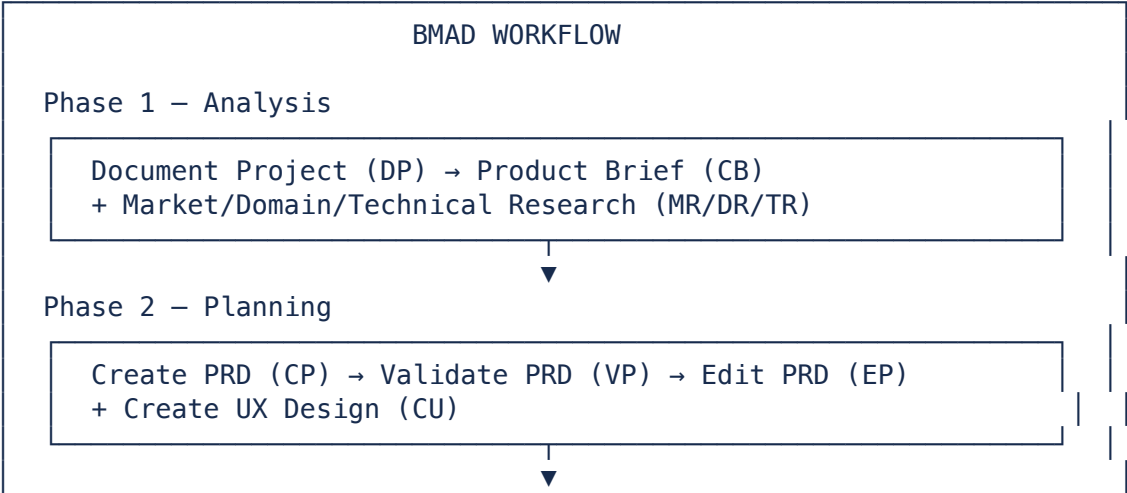
Skill	Menu	Description
bmad-create-story	CS	Prepare the next story item with full context for development
bmad-dev-story	DS	Execute story implementation (code + tests)
bmad-code-review	CR	Adversarial multi-layer code review (Blind Hunter, Edge Case Hunter, Acceptance Auditor)
bmad-qa-generate-e2e-tests	QA	Generate automated API and E2E test suites
bmad-quick-dev	QQ	Unified intent → plan → implement → review workflow
bmad-retrospective	ER	Post-epic review: lessons learned and next steps
bmad-correct-course	CC	Handle significant mid-sprint changes or pivots

Utility / Anytime Skills

Skill	Description
bmad-agent-tech-writer + write	Write any new technical document
bmad-agent-tech-writer + validate	Review a document against documentation standards
bmad-agent-tech-writer + mermaid	Generate a Mermaid architecture/flow diagram
bmad-agent-tech-writer + explain	Explain a complex technical concept with examples

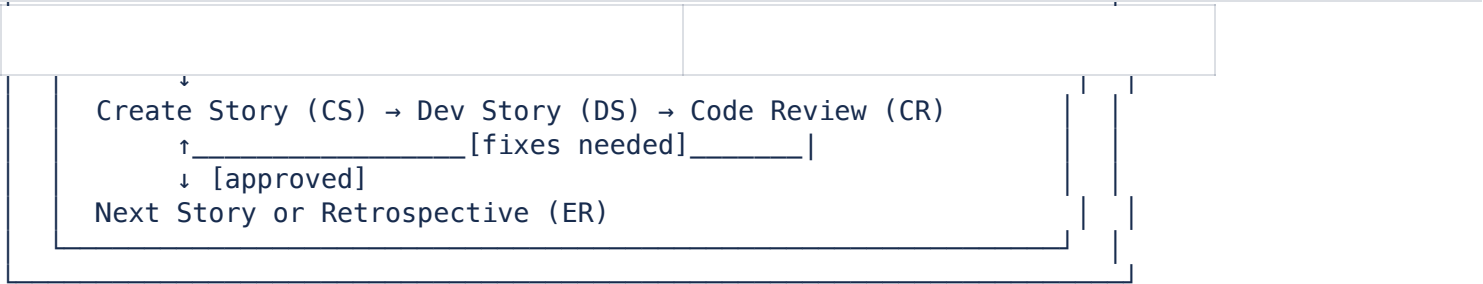
8. Full Development Workflow

The BMad Method follows a linear 4-phase workflow. Each phase produces artifacts consumed by the next.



Phase 3 – Solutioning

Create Architecture (CA) → Create Epics & Stories (CE)



Workflow Decision Points

Situation	Recommended Skill
New project idea, no requirements yet	bmad-product-brief → bmad-create-prd
Existing project, need AI context	bmad-document-project → bmad-generate-project-context
Fast solo feature build	bmad-quick-dev (Barry – bypasses full ceremony)
Mid-sprint requirement change	bmad-correct-course
Need a Mermaid diagram	bmad-agent-tech-writer with mermaid action
Post-epic review	bmad-retrospective

9. Use Cases & Examples

Use Case 1 – Brownfield Project Onboarding

Scenario: A developer joins the hw-backend-service project and wants AI to be context-aware.

Steps:

- 1. Install BMAD: `npx bmad-method@latest install`
- 2. In GitHub Copilot chat: `generate project context`
- 3. BMAD scans the codebase and produces `_bmad-output/project-context.md`
- 4. All subsequent AI requests automatically respect project conventions (DI style, logging, security config, etc.)

Output: `_bmad-output/project-context.md` — 76 rules covering technology stack, Spring patterns, testing, security, and anti-patterns.

Use Case 2 – New Feature Development (Full Workflow)

Scenario: Team wants to add a new Task Notification feature to hw-backend-service.

Steps:

Step	Skill	Who	Output
1	talk to John (Product	PM	Interview session to define requirements
	architecture		artifacts/ architecture.md
4	create the epics and stories list	Bob/SM	Epics & stories in planning-artifacts/
5	check implementation readiness	Auto	Readiness report with any gaps
6	run sprint planning	Bob/SM	_bmad-output/implementation-artifacts/sprint-status.md
7	create the next story	Bob/SM	Story file with full AC and context
8	dev this story	Amelia/Dev	Implemented code + unit tests
9	run code review	Quinn/QA	Code review report (issues → back to step 8)
10	create qa automated tests	Quinn/QA	E2E test suite
11	run a retrospective	Bob/SM	Lessons learned document

Use Case 3 – Quick Feature Implementation (Solo Dev)

Scenario: Developer needs to add a new REST endpoint quickly without full ceremony.

Trigger: lets build [feature description] or talk to Barry

Steps:

1. Barry (Quick Flow Solo Dev) clarifies the intent in 2–3 questions
2. Creates a lean tech spec inline
3. Implements the feature following existing project patterns
4. Runs a self-review
5. Presents the diff

Best for: Bug fixes, small enhancements, POCs, or when PRD/architecture already exists.

Use Case 4 – Code Review Automation

Scenario: A pull request is ready for review.

Trigger: run code review or review this code

What BMAD does:

- **Blind Hunter Layer:** Reviews code without looking at requirements — finds obvious bugs, security issues, style violations
- **Edge Case Hunter Layer:** Walks every branch and boundary condition
- **Acceptance Auditor Layer:** Validates code against story acceptance criteria

Output: Structured triage report with findings categorized as: Must Fix / Should Fix / Consider.

Use Case 5 – Documentation Generation

Scenario: Need to create or update technical documentation.

talk to Paige → mermaid [describe the flow/system]

Validate existing documentation
talk to Paige → validate [path/to/doc.md]

Explain a technical concept with examples
talk to Paige → explain [concept name]

Use Case 6 – Sprint Health Check

Scenario: Scrum Master or Tech Lead wants a quick view of sprint progress.

Trigger: check sprint status

Output: Summary including:

- Stories completed vs. in-progress vs. blocked
- Estimated completion
- Risks and blockers surfaced
- Recommended next action

10. Configuration Reference

_bmad/bmm/config.yaml

Controls the BMM module behavior for your specific project.

```
project_name: hw-backend-service
user_skill_level: intermediate      # beginner | intermediate | expert
planning_artifacts: "{project-root}/_bmad-output/planning-artifacts"
implementation_artifacts: "{project-root}/_bmad-output/implementation-artifacts"
project_knowledge: "{project-root}/docs"
user_name: Bdilipmohite
communication_language: English
document_output_language: English
output_folder: "{project-root}/_bmad-output"
```

Property	Options	Effect
user_skill_level	beginner / intermediate / expert	Controls verbosity of agent responses and amount of hand-holding
planning_artifacts	Any path	Where PRDs, architecture docs, epics are saved
implementation_artifacts	Any path	Where sprint plans, story files, test suites are saved

project knowledge	Any path	Reference documentation the AI reads

		Immediately on the next
skill invocation — no restart required.		

11. How Skills are Invoked

Skills registered in `.github/skills/` are automatically surfaced by GitHub Copilot Chat/ OpenAI ChatGPT. You trigger them with natural language:

Natural Language	Skill Invoked
help me brainstorm	bmad-brainstorming
create a product requirements document	bmad-create-prd
create technical architecture	bmad-create-architecture
create the epics and stories list	bmad-create-epics-and-stories
run sprint planning	bmad-sprint-planning
dev this story [file]	bmad-dev-story
run code review	bmad-code-review
generate project context	bmad-generate-project-context
document this project	bmad-document-project
check sprint status	bmad-sprint-status
bmad help	bmad-help

Tips for Effective Use

1. **Always generate project context first** on a new repo — it makes every subsequent skill dramatically more accurate.
2. **Commit `_bmad-output/`** to version control so the whole team shares the same AI context.
3. **Reference story files** when using `bmad-dev-story`: `dev this story _bmad-output/implementation-artifacts/story-1.1.md`
4. **Use `bmad help`** when you're unsure what to do next — it reads your current artifacts and recommends the next step.
5. **Party mode** (party mode) enables multi-agent conversations where multiple personas debate a design decision together.

12. Folder Conventions & Outputs

Planning Artifacts (`bmad-output/planning-artifacts/`)

		Document
architecture.md	<code>bmad-create-architecture</code>	Technical architecture and decisions
epics-and-stories.md	<code>bmad-create-epics-and-stories</code>	All epics, stories, and acceptance criteria
ux-design.md	<code>bmad-create-ux-design</code>	UX specifications and interaction patterns

Implementation Artifacts (`_bmad-output/implementation-artifacts/`)

File	Produced By	Description
sprint-status.md	<code>bmad-sprint-planning</code>	Sprint plan with story tracking
<code>story-[n.m].md</code>	<code>bmad-create-story</code>	Individual story file with full dev context
<code>retrospective-*.md</code>	<code>bmad-retrospective</code>	Post-epic lessons learned
<code>test-suite-*.md</code>	<code>bmad-qa-generate-e2e-tests</code>	Generated E2E/API test suite

Project Knowledge (docs/)

The docs/ folder is designated as `project_knowledge` — BMAD agents read this before acting. Keep it up to date with:

- API contracts (`docs/api/`)
- Architecture decisions (`docs/architecture/`)
- Deployment guides (`docs/guides/`)
- Analysis reports (`docs/analysis/`)

References

Resource	Link
BMAD GitHub Repository	https://github.com/bmadcode/bmad-method
Getting started with BMAD	https://docs.bmad-method.org/tutorials/getting-started/
BMAD Method guide	https://bmadmethodguide.com/bmadcode/BMAD-METHOD DeepWiki

	   Watch on
Initialization Recording	BenefitsIQ™ - BMAD installation setup -20260330_114638
Daily discussion Recording	Session 1: BenefitsIQ™ - BMAD Team Sync-20260330_130632-Meeting Recording Session 2: BenefitsIQ™ - BMAD Team Sync-20260331_130425-Meeting Recording Session 3: BenefitsIQ™ - BMAD Team Sync-20260401_130609-Meeting Recording Session 4: BenefitsIQ™ - BMAD Team Sync-20260402_130555-Meeting Recording Session 5: BenefitsIQ™ - BMAD Team Sync-20260406_130600-Meeting Recording Session 6: BenefitsIQ™ - BMAD Team Sync-20260407_131136-Meeting Recording Session 7: BenefitsIQ™ - BMAD Team Sync-20260414_131041-Meeting Recording Session 8: BenefitsIQ™ - BMAD Team Sync-20260415_130816-Meeting Recording Session 9: BenefitsIQ™ - BMAD Team Sync-20260420_130715-Meeting Recording

Document generated for the hw-backend-service project · BMAD v6.2.2 · March 30, 2026

No labels